

Trust-free Personalized Decentralized Learning

Yawen Li, Yan Li, Junping Du, Yingxia Shao, Meiyu Liang and Guanhua Ye*

Beijing University of Posts and Telecommunications, Beijing, China

{warmly0716, liyanly, junpingd, shaoyx, meiyu1210, g.ye}@bupt.edu.cn

Abstract—Personalized collaborative learning in federated settings faces a critical trade-off between customization and participant trust. Existing approaches typically rely on centralized coordinators or trusted peer groups, limiting their applicability in open, trust-averse environments. While recent decentralized methods explore anonymous knowledge sharing, they often lack global scalability and robust mechanisms against malicious peers. To bridge this gap, we propose TPFed, a *Trust-free Personalized Decentralized Federated Learning* framework. TPFed replaces central aggregators with a blockchain-based bulletin board, enabling participants to dynamically select global communication partners based on Locality-Sensitive Hashing (LSH) and peer ranking. Crucially, we introduce an “all-in-one” knowledge distillation protocol that simultaneously handles knowledge transfer, model quality evaluation, and similarity verification via a public reference dataset. This design ensures secure, globally personalized collaboration without exposing local models or data. Extensive experiments demonstrate that TPFed significantly outperforms traditional federated baselines in both learning accuracy and system robustness against adversarial attacks.

Index Terms—Federated Learning, Personalized Decentralized Learning, Trust-free Collaboration, Knowledge Distillation.

I. INTRODUCTION

The proliferation of distributed devices has established Federated Learning (FL) [1] as a standard for privacy-preserving intelligence. While conventional Centralized FL (CFed) aggregates updates globally, it often struggles with heterogeneous, non-IID data distributions common in real-world scenarios [2]. Consequently, personalized FL has emerged, where clients adapt a global model to local distinctiveness. Recent studies emphasize that constructing a *communication graph*—allowing clients to selectively exchange information only with high-quality, similar neighbors—yields superior personalized outcomes [3].

However, implementing such selective collaboration without a trusted third party remains a significant challenge. Existing Semi-decentralized (SFed) methods [4] employ a central coordinator to manage topology, introducing a single point of failure and inherent trust assumptions. Conversely, fully Decentralized (DFed) approaches [5] typically rely on local gossip protocols. These often suffer from limited global visibility and lack rigorous mechanisms to verify peer honesty, leaving the system vulnerable to poisoning or free-riding behaviors.

To address these limitations, we propose **TPFed** (*Trust-free Personalized Decentralized Federated Learning*). TPFed adheres to a strict trust-free design philosophy: no single entity possesses control, and no participant is assumed

to be inherently trustworthy. Unlike conventional gossip-based schemes (DFed(G)), TPFed introduces a “Web-based” paradigm (DFed(W)) that enables global neighbor discovery and verification through three integrated mechanisms:

All-in-one P2P Knowledge Distillation. TPFed extends knowledge distillation to a fully decentralized setting, allowing each client to perform knowledge transfer, model quality evaluation, and inter-client similarity measurement *in one shot*. Specifically, each client requests from its peers the predicted logits for a locally held (and privacy-preserving) reference dataset [6], [7]. This eliminates the need for trusting a central aggregator with raw model updates or sensitive data, ensuring the process is inherently trust-free.

Immutable Bulletin Board for Coordination. Instead of relying on a centralized coordinator, TPFed leverages a public immutable ledger (e.g., a smart contract or a Layer-2 blockchain solution) to serve as a global bulletin board. This guarantees that no third party can manipulate or censor posted bulletins. To ensure practical scalability and low latency, we adopt an asynchronous update scheme: clients retrieve neighbor information from the ledger but only commit their own updates periodically (e.g., every K rounds) or when significant model changes occur. By simply reading the transparent ledger, each client discovers with whom to communicate via Locality-Sensitive Hashing (LSH) [8], [9] and Peer Ranking (PR) mechanisms. This design removes the single point of failure while maintaining a trust-free coordination layer.

Countermeasures Against Malicious Behaviors. The trust-free nature of open networks invites adversaries to forge LSH codes or collude. TPFed secures the system via a commit-then-reveal ranking scheme and a fine-grained consistency check. By comparing a peer’s claimed LSH distance against the actual divergence of their logits, clients can locally detect and blacklist malicious actors who attempt to spoof similarity or degrade performance.

In light of this, we differentiate TPFed from other fully decentralized federated learning approaches. In the sections that follow, we refer to conventional decentralized FL reliant on Gossip-based communication as DFed(G), whereas our TPFed framework is denoted as DFed(W), reflecting its ability to retrieve blockchain information via the **Web** and perform direct peer-to-peer exchanges with any node. We believe this Web3.0-oriented FL paradigm holds significant potential for future research. Our main contributions are summarized as follows:

- We identify the gap in achieving robust *model quality* and *similarity evaluation* in fully decentralized, trust-free FL

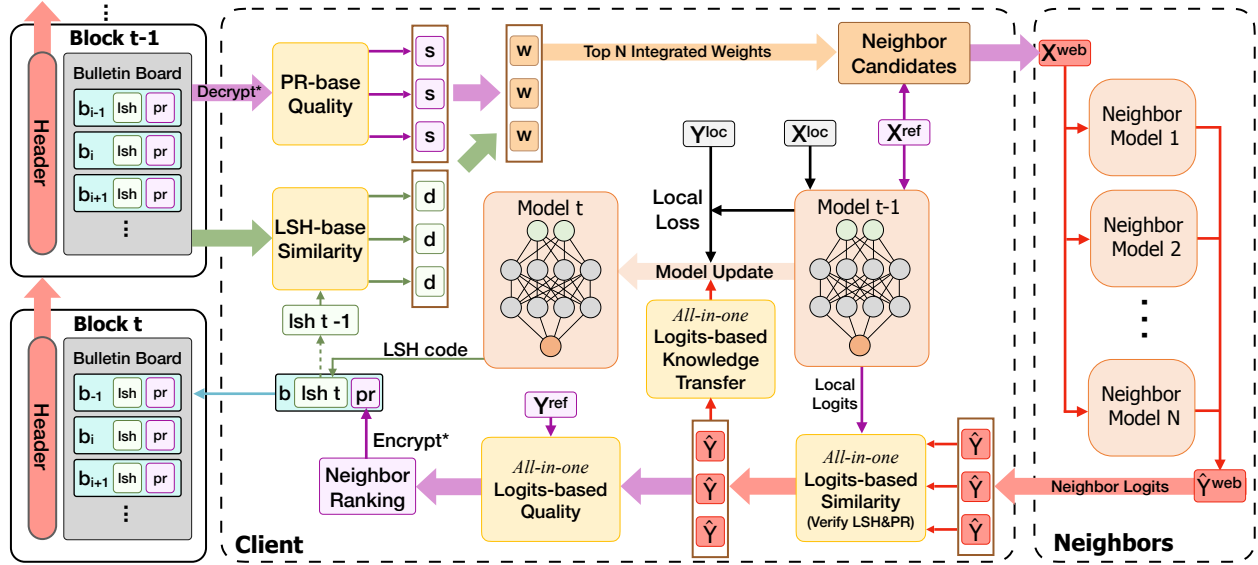


Fig. 1. **Overview of the TPFed framework.** The central coordinator is replaced by a blockchain-based bulletin board, which filters neighbor candidates based on inter-client similarity (via *LSH*) and model quality (via *PR*). Clients then employ *all-in-one logits* to simultaneously perform knowledge transfer, model quality evaluation, and inter-client similarity measurement, where the latter also verifies the *LSH* and *PR* claims. *In practice, clients publish commitments C in the bulletin instead of the original neighbor ranking PR ; however, PR is depicted in the figure to simplify the illustration of data flow.

environments.

- We propose TPFed, a novel framework combining blockchain-based coordination with all-in-one knowledge distillation to enable secure, global-scale neighbor selection.
- Comprehensive experiments on real-world datasets demonstrate that TPFed substantially outperforms existing baselines in accuracy and resilience against DoS and poisoning attacks.

II. RELATED WORK

Decentralized Federated Learning. Decentralized Federated Learning (DFL) mitigates the single-point-of-failure risks associated with centralized servers by enabling direct client-to-client coordination [10]. This architecture is particularly advantageous for privacy-sensitive and communication-constrained environments. However, the majority of DFL approaches rely on static or locally restricted topologies, such as physical proximity or random gossip protocols. While these methods function in stable settings, they often underutilize global network information. Consequently, they lack the adaptability required for dynamic environments where optimal collaboration partners may not be topologically adjacent [11].

Personalized Neighbor Selection Mechanism. Effective neighbor selection is pivotal for achieving personalization and fast convergence in DFL [12]. Conventional solutions typically employ heuristic strategies, such as location-based clustering [13] or random sampling [14], which fail to account for the semantic similarity between models. To address this, recent works have leveraged Locality-Sensitive Hashing (LSH) to estimate model similarity while preserving privacy [9]. Nevertheless, relying exclusively on LSH without verifying model quality or preventing adversarial manipulation remains

insufficient for ensuring robust personalization in open networks.

Web-based Collaborative Learning. The evolution of web technologies, including WebRTC [15], Distributed Hash Tables (DHTs) [16], and Decentralized Identifiers (DIDs) [17], has paved the way for serverless, real-time collaboration. These open architectures, however, introduce significant trust and security challenges, prompting a shift toward blockchain-facilitated frameworks to enforce accountability [18]. Furthermore, to safeguard data during these interactions, cryptographic techniques such as differential privacy [19] and secure multi-party computation [20] are increasingly integrated into collaborative pipelines.

III. PRELIMINARIES

This section introduces the key concepts and components of TPFed: user clients, bulletins, peer-to-peer (P2P) communication, and the system-wide objective.

A. User Clients

Each user i corresponds to a client u_i that maintains a local dataset $(\mathbf{X}_i^{\text{loc}}, \mathbf{Y}_i^{\text{loc}})$, a reference dataset $(\mathbf{X}_i^{\text{ref}}, \mathbf{Y}_i^{\text{ref}})$, and local parameters θ_i . The inference of model θ_j is $f(\theta_j, \cdot)$. The reference dataset facilitates communication with others without exposing sensitive local data. Each client manages a dynamic set of neighbors $\mathcal{N}_i$, which may change over time. Communication with these neighbors occurs in a P2P manner.

B. Bulletins

Each client publicizes a *bulletin* $\mathbf{b}_i^{(t)} = \{\text{lsh}_i^{(t)}, \mathbf{R}_i^{(t)}\}$ on the blockchain, where $\text{lsh}_i^{(t)}$ is a locality-sensitive hash (LSH) of the model at iteration t , and $\mathbf{R}_i^{(t)}$ ranks its neighbors based

on performance evaluations with the reference dataset ¹. From the bulletins $\{\mathbf{b}_j^{(t)} | j \neq i\}$, client u_i infers similarity scores $\{d_{ij}\}$ from the LSH codes and ranking scores $\{s_j\}$ from peer performance. This eliminates the need for a central coordinator and allows autonomous neighbor selection.

C. Dynamic Communication Graph

The overall network is a dynamic graph $G = (U, E, W)$ with $U = \{u_i | i \in [1, M]\}$, where M is the total number of clients. An edge $e_{ij} = 1$ indicates direct P2P connectivity between u_i and u_j , and w_{ij} represents the potential benefit of their communication. Typically, all clients can communicate ($e_{ij} = 1$), while w_{ij} is derived from bulletins and reflects model similarity and peer performance.

D. TPFed Objective

TPFed aims to help each client find an optimal neighbor set $\mathcal{N}_i$ by leveraging W to improve overall performance. The global objective is:

$$\mathcal{L}_{\text{global}}^* = \frac{1}{M} \sum_{i=1}^M \left[\alpha \ell(f(\theta_i, \mathbf{X}_i^{\text{loc}}), \mathbf{Y}_i^{\text{loc}}) + (1 - \alpha) \mathcal{L}_i^{\text{ref}} \right], \quad (1)$$

where $\mathcal{L}_i^{\text{ref}}$ measures alignment with neighbors on $\mathbf{X}_i^{\text{ref}}$, $\ell(\cdot)$ is the loss function, and $\alpha \in [0, 1]$ controls the balance between local and collaborative terms. By minimizing $\mathcal{L}_{\text{global}}^*$, TPFed encourages robust local performance while promoting effective cooperation across the network.

IV. METHODOLOGY

This section details the TPFed framework. The complete workflow is summarized in Algorithm 1.

A. All-in-one P2P Knowledge Distillation

Knowledge Transfer. Clients in TPFed communicate directly to refine local models via knowledge distillation. To strictly preserve privacy, the data exchanged during this process comes from a *public, non-sensitive reference dataset* $\mathcal{D}^{\text{ref}}$, which is disjoint from any client's private local data. As shown in Fig. 2, the process starts with client u_i sharing the features $\mathbf{X}_i^{\text{ref}}$ (sampled from $\mathcal{D}^{\text{ref}}$) with neighbor u_j , which responds with $\hat{\mathbf{Y}}_j^{\text{web}} = f(\theta_j, \mathbf{X}_i^{\text{ref}})$. Although $\mathbf{X}_i^{\text{ref}}$ follows a public distribution different from the private non-IID data, it serves as a semantic anchor [21]. If two models produce similar logits on a diverse public dataset, they likely share compatible decision boundaries, making knowledge transfer effective even across heterogeneous domains. [22] Client u_i , having the ground truth $\mathbf{Y}_i^{\text{ref}}$ (from the public source), assesses u_j 's performance and updates θ_i by aligning $f(\theta_i, \mathbf{X}_i^{\text{ref}})$ with these logits. When multiple neighbors participate, the update aggregates all $\hat{\mathbf{Y}}_j^{\text{web}}$:

$$\mathcal{L}_i^{\text{ref}} = \left\| f(\theta_i, \mathbf{X}_i^{\text{ref}}) - \frac{1}{N} \sum_{j=1}^N \hat{\mathbf{Y}}_j^{\text{web}} \right\|^2. \quad (2)$$

¹Practically implemented as a commitment $C_i^{(t)}$ (see Sec. IV-C); $\mathbf{R}_i^{(t)}$ is used here to simplify the scoring formulation.

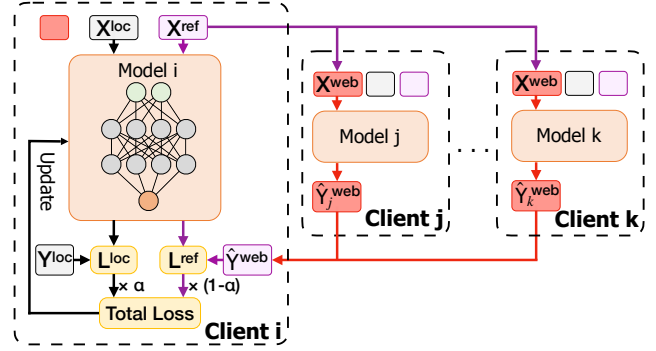


Fig. 2. Client learns from neighbors via collecting logits $\hat{\mathbf{Y}}^{\text{web}}$.

All-in-one nature. In a single exchange of logits, TPFed simultaneously achieves knowledge transfer, peer quality measurement, and similarity verification. By comparing each neighbor's $\hat{\mathbf{Y}}_j^{\text{web}}$ against $\mathbf{Y}_i^{\text{ref}}$, the client quantifies that neighbor's model quality, while examining how well different logits align with the local predictions provides a measure of inter-model similarity. This approach keeps reference data private and cuts communication overhead, as the same logits serve both local updates and evaluation needs.

B. Personalization via Bulletin Board

Semi-decentralized methods benefit from a globally informed coordinator that can select neighbors based on inter-client similarity and model quality. In TPFed, we replace this centralized coordinator with a blockchain-based bulletin board to achieve a similar effect.

LSH-based similarity. Each client posts a *locality-sensitive hash* (LSH) code derived from its model parameters. To mitigate the high cost and latency associated with frequent blockchain interactions, clients are not required to update their LSH codes in every iteration. Instead, we implement an *asynchronous update strategy* with an interval T_{sync} . Client u_i computes and publishes $\text{ls}_i = \text{LSH}(\theta_i)$ every T_{sync} rounds. Between updates, peers rely on the most recent on-chain snapshot for neighbor selection. Models that are highly similar generate hash codes that match with high probability. The Hamming distance between two codes, $d_{ij} = \text{HammingDist}(\text{ls}_i, \text{ls}_j)$, reflects their approximate similarity; a smaller d_{ij} indicates closer models. It is worth noting that while neural networks are permutation invariant, we adopt the standard FL assumption of a shared initialization $\theta^{(0)}$. This ensures that client models remain in a broadly aligned parameter space during training, making LSH on parameters a computationally efficient proxy for functional similarity. Even in cases where permutation leads to falsely large distances, TPFed's redundancy allows clients to find other sufficient neighbors, while the subsequent logits verification prevents false positives.

$$d_{ij} = \text{HammingDist}(\text{ls}_i, \text{ls}_j), \quad (3)$$

reflects their approximate similarity; a smaller d_{ij} indicates closer models. It is worth noting that while neural networks are permutation invariant, we adopt the standard FL assumption of a shared initialization $\theta^{(0)}$. This ensures that client models remain in a broadly aligned parameter space during training, making LSH on parameters a computationally efficient proxy for functional similarity. Even in cases where permutation leads to falsely large distances, TPFed's redundancy allows clients to find other sufficient neighbors, while the subsequent logits verification prevents false positives.

PR-based quality. Each client also publishes a Peer Ranking (PR) based on the peer losses observed during P2P communication. For example, u_i produces an ordered list

Algorithm 1 TPFed Framework for Client u_i

Require: Local $\{\mathbf{X}_i^{\text{loc}}, \mathbf{Y}_i^{\text{loc}}\}$, Reference $\{\mathbf{X}_i^{\text{ref}}, \mathbf{Y}_i^{\text{ref}}\}$, Init θ_i , Iterations T , Hyperparams $(\alpha_i, \gamma, \epsilon)$, Neighbors N

- 1: **for** $t = 1$ **to** T **do**
- 2: **Neighbor selection**
- 3: **if** $t > 1$ **then**
- 4: Retrieve $\mathbf{b}_j^{(t-1)} = \{\text{lsh}_j^{(t-1)}, C_j^{(t-1)}\}$ and $\mathbf{R}_j^{(t-1)}$; verify $\mathbf{R}_j^{(t-1)}$
- 5: **end if**
- 6: For each u_j : compute d_{ij} (Eq. (2)) and s_j (Eq. (3))
- 7: $w_{ij} \leftarrow s_j \exp(-\gamma d_{ij})$; $\mathcal{N}_i^{(t)} \leftarrow \text{Top}-(1-\epsilon)N$ by $w_{ij} \cup \text{Random-}\epsilon N$
- 8: **Communication**
- 9: **for** $u_j \in \mathcal{N}_i^{(t)}$ **do**
- 10: Send $\mathbf{X}_i^{\text{ref}}$ to u_j , receive $f(\theta_j^{(t)}, \mathbf{X}_i^{\text{ref}})$
- 11: $\ell_{ij}^{(t)} \leftarrow \ell(f(\theta_j^{(t)}, \mathbf{X}_i^{\text{ref}}), \mathbf{Y}_i^{\text{ref}})$
- 12: **end for**
- 13: **Model update**
- 14: Calculate weights $p_{ij} \propto w_{ij}^{\text{final}}$ (Eq. 6) using $\ell_{ij}^{(t)}$ and consistency $\mathcal{V}_{ij}$
- 15: $\bar{f}^{(t)} \leftarrow \sum_{u_j \in \mathcal{N}_i^{(t)}} p_{ij} \cdot f(\theta_j^{(t)}, \mathbf{X}_i^{\text{ref}})$
- 16: $\mathcal{L}^{(t)} \leftarrow \alpha_i \ell(f(\theta, \mathbf{X}_i^{\text{loc}}), \mathbf{Y}_i^{\text{loc}}) + (1 - \alpha_i) \|f(\theta, \mathbf{X}_i^{\text{ref}}) - \bar{f}^{(t)}\|^2$
- 17: $\theta_i^{(t+1)} \leftarrow \text{Update}(\theta_i^{(t)}, \nabla_{\theta} \mathcal{L}^{(t)})$
- 18: **Bulletin publication**
- 19: $\text{lsh}_i^{(t)} \leftarrow \text{update via Eq. (4)}; \mathbf{R}_i^{(t)} \leftarrow \text{sort by ascending } \ell_{ij}^{(t)}$
- 20: $C_i^{(t)} \leftarrow \text{Hash}(\mathbf{R}_i^{(t)})$; publish $\mathbf{b}_i^{(t)} = \{\text{lsh}_i^{(t)}, C_i^{(t)}\}$
- 21: Reveal $\mathbf{R}_i^{(t)}$ for verification at $t+1$
- 22: **end for**

$\mathbf{R}_i$, where higher-ranked neighbors have lower loss ℓ_{ij} . By aggregating these rankings from all clients, each peer u_j is assigned a *ranking score*:

$$s_j = \frac{|\{\mathbf{R}_k \mid u_j \in \text{top-}K(\mathbf{R}_k), k \in [1, M]\}|}{|\{\mathbf{R}_k \mid u_j \in \mathbf{R}_k, k \in [1, M]\}|} \quad (4)$$

This score captures how often u_j is a top- K performer across the network.

Personalized neighbor selection. Each client combines the ranking score s_j and similarity distance d_{ij} to derive a weight:

$$w_{ij} = s_j \cdot \exp(-\gamma \cdot d_{ij}), \quad (5)$$

where γ is a hyperparameter controlling the emphasis on similarity. A higher weight indicates a neighbor that is both well-ranked and closely aligned in model space. Client u_i then selects the top N peers with the largest w_{ij} values to form its neighbor set $\mathcal{N}_i$, ensuring collaborations are tailored to each client’s unique observations and requirements.

Exploration Mechanism. Relying strictly on w_{ij} may prevent unranked peers from ever being evaluated (the “cold start” problem). To address this, we employ an ϵ -greedy strategy: client u_i selects $(1 - \epsilon)N$ neighbors based on the highest weights (exploitation) while randomly sampling the remaining ϵN candidates from the network.

C. Countermeasures Against Malicious Behaviors

The trust-free nature of TPFed introduces two primary attack vectors: *Sybil/DoS attacks* via LSH forgery, and *Collusion/Ranking manipulation* in peer reviews. We address these through a rigorous verify-and-penalize mechanism that leverages the intrinsic workflows of TPFed.

LSH Forgery & DoS. Adversaries may forge LSH codes to minimize the Hamming distance d_{ij} (Eq. 3), tricking client u_i into selecting them as neighbors. A naive system would be vulnerable to Denial-of-Service (DoS) attacks, where checking massive numbers of fake candidates exhausts u_i ’s bandwidth. We repurpose this existing data flow to perform a *strict consistency check*. Upon receiving logits $\hat{\mathbf{Y}}_j^{\text{web}}$ from a selected neighbor u_j , client u_i computes the actual semantic divergence D_{KL} and compares it with the claimed LSH distance:

$$\mathcal{V}_{ij} = \mathbb{I} \left(\left| \frac{d_{ij}}{L} - \phi(D_{\text{KL}}(\mathbf{Y}_i^{\text{ref}} \parallel \hat{\mathbf{Y}}_j^{\text{web}})) \right| \leq \tau \right), \quad (6)$$

where L is the hash length, $\phi(\cdot)$ is a normalizing mapping function, and τ is a tolerance threshold. If $\mathcal{V}_{ij} = 0$ (check failed), it implies u_j ’s LSH code contradicts its actual model behavior. To prevent DoS recurrence, u_i immediately adds u_j to a **Local Blacklist** for T_{ban} rounds. This creates an *asymmetric cost structure*: attackers must pay real-world blockchain transaction fees to continuously register new identities (LSHs) to sustain an attack, whereas honest clients filter them out with a one-time verification cost.

Collusion. Malicious cliques may engage in *collusion*, mutually boosting each other’s ranking scores s_j . **Countermeasure:** In Sec. IV-B, we introduced a **commit-and-reveal** scheme (publishing hash commitments $C_i^{(t)}$) to prevent clients from altering their evaluations after seeing others’. While this prevents ex-post modification, it cannot verify the *honesty* of the ranking source itself (e.g., pre-planned collusion). TPFed resolves this by treating the global ranking score s_j merely as a *discovery heuristic*, not a ground truth. The final decision weight w_{ij} is dominated by the *local* evaluation of the exchanged logits. Even if a malicious node u_j achieves a high s_j through collusion, its effective weight is dynamically adjusted:

$$w_{ij}^{\text{final}} = \mathcal{V}_{ij} \cdot w_{ij} \cdot \exp(-\lambda \cdot \mathcal{L}_i^{\text{ref}}(u_j)). \quad (7)$$

If the locally measured loss $\mathcal{L}_i^{\text{ref}}(u_j)$ is high (indicating poor alignment with u_i ’s objective), the neighbor is effectively discarded regardless of its inflated global reputation. This ensures that collusion attacks can at most affect the *candidate selection* phase but cannot corrupt the actual *model update*.

Reference Overfitting. Besides, to prevent advanced adversaries from overfitting to the static public reference dataset $\mathbf{X}^{\text{ref}}$ (i.e., memorizing answers to pass checks while holding a corrupted model), clients employ *Randomized Querying*. In each round, u_i applies random augmentations (e.g., cropping, noise injection) to $\mathbf{X}^{\text{ref}}$ before sending it to neighbors. This forces the adversary to provide logits that are robust to unseen perturbations. Crucially, if an adversary possesses a model robust enough to handle these random augmentations consistently, their contribution becomes mathematically equivalent to that of a benign, high-quality teacher. In this scenario, the attack becomes self-defeating: to pass the verification, the adversary is forced to behave honestly and contribute valid knowledge, thereby nullifying the intent to degrade the victim’s performance.

V. EXPERIMENTS

In this section, we conduct comprehensive experiments to evaluate the TPFed framework. We structure our evaluation to answer the following Research Questions (RQs):

- **RQ1 (Effectiveness):** Does TPFed outperform existing centralized, semi-decentralized, and decentralized federated learning baselines in terms of model accuracy?
- **RQ2 (Identity Defense):** Can TPFed effectively detect and reject adversaries attempting to spoof model similarity (LSH forgery) to perform impersonation attacks?
- **RQ3 (Poisoning Resilience):** How robust is TPFed against active poisoning attacks where malicious clients inject noise to disrupt the training process?
- **RQ4 (Scalability):** How does the communication overhead of TPFed compare to other methods, and does the TPFed-idx variant effectively improve scalability?
- **RQ5 (Ablation):** What is the contribution of each core component (LSH, Peer Ranking, and Logits Verification) to the overall framework performance?
- **RQ6 (System Robustness):** Can TPFed maintain stability under adverse system conditions, such as network delays, transmission failures, or low-quality reference data?

A. Baselines

We compare **TPFed** with four representative baselines covering the spectrum from isolated training to fully decentralized learning.

SILO [23]. Devices train solely on local data without any interaction. This maximizes privacy but loses the benefits of cross-device collaboration, typically yielding weaker models.

Centralized (CFed). We include **BlockFL** [24] and **FedMD** [25]. BlockFL employs a blockchain-backed mechanism to share gradients and build a global model but still depends on a central coordinator. FedMD aggregates heterogeneous models via distillation using a public or proxy dataset under centralized orchestration.

Semi-Decentralized (SFed). We evaluate **pFedClub** [26] and **SQMD** [27]. pFedClub personalizes models through functional block decomposition, while SQMD manages asynchronous participation and device heterogeneity using a central communication graph. Both achieve strong performance but require trusted coordinators, limiting robustness in fully decentralized environments.

Decentralized (DFed(G)). We consider **ProxyFL** [28] and **KD-PDFL** [29]. ProxyFL exchanges proxy models for enhanced privacy, and KD-PDFL uses knowledge distillation for peer collaboration. Although free of central aggregators, these methods still assume honest peer behavior and remain vulnerable to adversarial manipulation.

B. Datasets and Implementation Details

Datasets. We evaluate on four benchmarks: **MNIST** [30] (70,000 handwritten digits; the non-IID split is described under *Partitioning*); **A-ECG** [31] (35 overnight ECG recordings for

sleep apnea; signals preprocessed into 60-dimensional RR-interval vectors [32]); **S-EEG** [33], [34] (153 polysomnography recordings; we select 40 high-quality EEG records and perform three-class staging: awake, NREM, REM [34]); **CIFAR-100** [35] (100 classes grouped into 20 superclasses; we perform within-superclass classification—e.g., flowers, insects—to increase data heterogeneity and task difficulty).

Partitioning. Because TPFed targets personalized analysis, multi-subject datasets adopt a “one subject = one client” scheme, yielding 35 and 40 clients for A-ECG and S-EEG, respectively. For MNIST, which lacks subject identifiers, we simulate non-IID heterogeneity by randomly dividing the training set into 20 shards and assigning two shards to each of 10 clients, while removing one digit class per shard to induce class bias. To mitigate the limited sample size in A-ECG and S-EEG, we apply a sliding-window augmentation [32] to each subject’s recordings. Each client’s local data are finally split into training and test sets with a 7:3 ratio.

Reference Data. To enable trust-free communication without exposing private data, TPFed utilizes a shared public reference dataset $\mathbf{X}^{\text{ref}}$. In practical deployment, this data originates from publicly available repositories (e.g., open benchmarks like PhysioNet), ensuring it is non-sensitive and strictly disjoint from clients’ private data. In our experiments, to simulate this setting, we reserve 20% of the subjects from A-ECG and S-EEG to act as this public proxy data, while the remaining subjects serve as private clients. For MNIST, the standard test set (10,000 images) is used as the public reference repository. Each client uniformly samples a non-overlapping subset from this public repository to facilitate checking logits and calculating divergence.

Training Setup. All methods are implemented in PyTorch with a ResNet backbone [36]. The federated environment is simulated on a server with $4 \times$ NVIDIA V100 GPUs. The loss ℓ is cross-entropy for all devices, and performance is reported as accuracy (Acc) on all datasets.

C. Performance Evaluation (RQ1)

Table I reports the performance on four benchmarks. We also include **TPFed-idx**, a scalable variant detailed in Sec. V-F, to highlight its efficiency. Overall, TPFed achieves state-of-the-art accuracy, consistently outperforming semi-decentralized and gossip-based baselines. Its superiority in heterogeneous scenarios (e.g., S-EEG, CIFAR-100) confirms the robustness of our trust-free framework. While TPFed-idx exhibits a marginal accuracy trade-off, it still surpasses existing methods.

D. Resilience to LSH Cheating (RQ2)

RQ2 evaluates resilience against LSH Cheating, where adversaries forge codes to impersonate neighbors and inject malicious updates. In simulations (attack start $t = 50$), Fig. 3 shows that while unverified systems suffer sharp accuracy drops, TPFed remains stable. This robustness stems from our consistency check (Eq. 6): since attackers cannot generate logits semantically aligned with the victim’s private reference

TABLE I
PERFORMANCE COMPARISON OF DIFFERENT FRAMEWORKS ACROSS BASE MODELS AND DATASETS

Category	Framework	P-MNIST	A-ECG	S-EEG	CIFAR-100
SILO	SILO	0.8774 ± 0.0088	0.9112 ± 0.0050	0.8367 ± 0.0090	0.4210 ± 0.0145
CFed	BlockFL	0.9342 ± 0.0011	0.9117 ± 0.0016	0.8358 ± 0.0045	0.4510 ± 0.0028
	FedMD	0.9375 ± 0.0017	0.9116 ± 0.0016	0.8324 ± 0.0050	0.4630 ± 0.0020
SFed	pFedClub	0.9395 ± 0.0019	0.9176 ± 0.0021	0.8421 ± 0.0077	0.4805 ± 0.0041
	SQMD	0.9379 ± 0.0032	0.9124 ± 0.0030	0.8446 ± 0.0063	0.4850 ± 0.0035
DFed(G)	ProxyFL	0.9224 ± 0.0051	0.9051 ± 0.0066	0.8391 ± 0.0091	0.4701 ± 0.0053
	KD-PDFL	0.9232 ± 0.0033	0.9106 ± 0.0050	0.8444 ± 0.0088	0.4623 ± 0.0048
DFed(W)	TPFed-idx	0.9375 ± 0.0023	0.9156 ± 0.0059	0.8384 ± 0.0060	0.4937 ± 0.0042
	TPFed	0.9422 ± 0.0041	0.9287 ± 0.0038	0.8525 ± 0.0065	0.5295 ± 0.0050

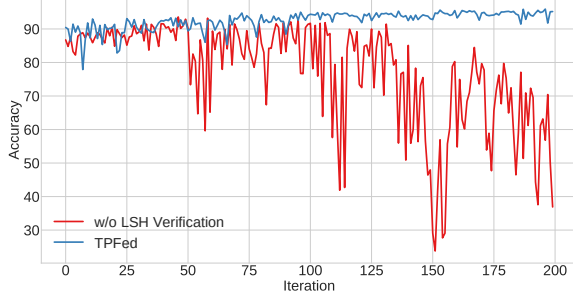


Fig. 3. One LSH Cheating Attack Start from $t = 50$.

data, their forged identities are effectively detected and rejected.

E. Resilience to Poisoning Attacks (RQ3)

We evaluated TPFed under a poisoning scenario where certain clients periodically reinitialized their model parameters after a warm-up phase, injecting noise to disrupt honest clients. Figure 4 compares the performance of TPFed and ProxyFL for varying proportions of malicious clients (20%, 40%, 60%). While ProxyFL suffers notable performance drops as the fraction of malicious clients grows, TPFed remains largely unaffected at the 20% and 40% levels and displays only moderate fluctuations at 60%. This resilience derives from TPFed’s global neighbor selection mechanism, wherein each client chooses high-performing (and therefore rarely malicious) peers based on both LSH similarity and rank-based evaluations, preventing malicious nodes from significantly influencing honest participants.

In all cases, the drop in accuracy remains modest, indicating that excluding delayed or missing bulletins does not drastically impair overall performance.

F. Scalability (RQ4)

We compare four representative methods across different categories in Table II. To strictly evaluate the algorithmic communication and computation costs without the volatility of public network congestion or gas fees, we conduct our simulations assuming the bulletin board is deployed on a high-throughput Layer-2 environment (as discussed in Sec. IV-B). Under this logical infrastructure, TPFed achieves a convergence speed on par with gossip-based decentralized learning

TABLE II
OVERHEAD OF FOUR REPRESENTATIVE METHODS (SET THE OVERHEAD OF FEDMD AS THE BASELINE).

Category	Method	Overhead per Round	Overhead at 40% Acc
CFed	FedMD	1.00	1.00
SFed	SQMD	1.05	1.10
DFed(G)	ProxyFL	1.08	1.22
DFed(W)	TPFed	1.25	1.21

TABLE III
COMPARISON OF TPFED AND TPFED-idx WITH $H=20$ ON CIFAR-100. (M IS THE NUMBER OF CLIENTS. H IS THE RETRIEVE RANGE. OVERHEAD PER ROUND IS NORMALIZED TO FEDMD)

M	TPFed Acc	TPFed Overhead	TPFed-idx Acc	TPFed-idx Overhead
20	0.4505	1.17	0.4505	1.17
100	0.5295	1.25	0.4937	1.17
500	0.5323	1.68	0.4991	1.17

but exhibits a higher Extra Overhead per Round ($1.25\times$) compared to baselines.

Therefore, we present a detailed theoretical analysis and introduce a scalable variant, **TPFed-idx**. Unlike the standard TPFed where every client parses the entire bulletin board, TPFed-idx keeps the training process decentralized while leveraging the blockchain’s global index as an *auxiliary service*. A smart contract pre-sorts the LSH codes to generate a global index, allowing clients to retrieve only the top- H relevant candidates ($H \ll M$). This reduces the client-side sorting complexity significantly. We evaluate both TPFed and TPFed-idx on CIFAR-100 and summarize the results in Table III.

G. Ablation Study (RQ5)

We conduct an ablation study to evaluate the contributions of each main component in **TPFed** by removing them individually. Specifically, we consider three variants: **w/o LSH** discards the locality-sensitive hashing approach, relying entirely on peer ranking (PR) to select neighbors. **w/o PR** excludes the ranking mechanism, basing neighbor selection solely on LSH-derived similarity. **w/o logits** removes the all-in-one logits exchange, so neither LSH nor PR can be verified during P2P interactions.

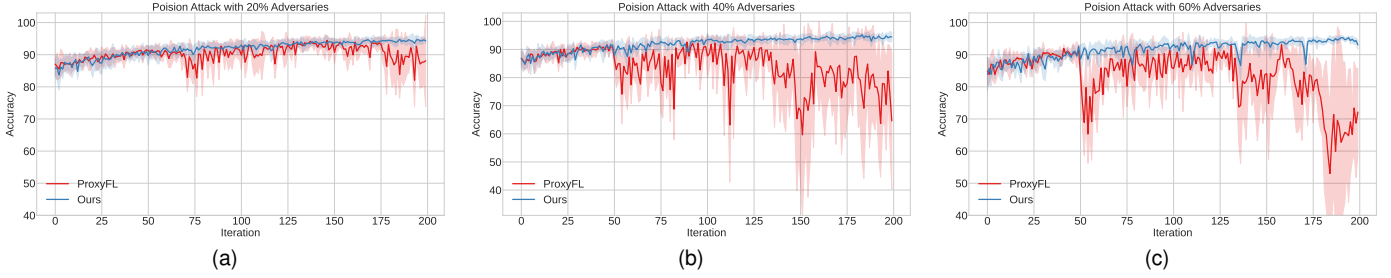


Fig. 4. Poison attacks with varying amounts of dishonest clients. Attacks start from the 50th iteration. Blue curve: TPFed; red curve: ProxyFL. Shaded areas denote the standard deviation across honest clients.

TABLE IV
ABLATION STUDY FOR TPFED

Config	MNIST	A-ECG	S-EEG	CIFAR
TPFed	0.9422	0.9287	0.8525	0.5295
w/o LSH	-0.0104	-0.0070	-0.0153	-0.0194
w/o PR	-0.0116	-0.0061	-0.0198	-0.0308
w/o logits	-0.0282	-0.0195	-0.0202	-0.0371

TABLE V
IMPACT OF DELAYED AND FAILED UPLOADS ON TPFED'S ACCURACY.

Dataset	20% Delay	40% Delay	20% Fail
MNIST	-0.0085	-0.0132	-0.0143
A-ECG	-0.0063	-0.0091	-0.0120
S-EEG	-0.0049	-0.0078	-0.0101
CIFAR	-0.0024	-0.0042	-0.0065

Table IV reports the performance differences of these variants relative to full TPFed. Omitting either LSH or PR leads to noticeable accuracy drops (about 1–3% across most datasets), confirming that both similarity checks and peer ranking are crucial for effective neighbor selection. Notably, **w/o logits** suffers the largest decrease in performance, highlighting the importance of verifying claims about model quality and similarity through the reference-data logits. This verification mechanism directly prevents adversarial behaviors and ensures that LSH codes and PR scores accurately reflect genuine model characteristics.

H. Abnormal Uploads (RQ6)

Although TPFed uses blockchain-based bulletins for decentralized coordination, real-world deployments may encounter latency, network congestion, or node misbehavior. We incorporate a *timeout* mechanism: if a client's bulletin fails to appear on-chain before a certain deadline, other clients skip its data in that round. Table V shows three scenarios where 20% or 40% of clients delay their uploads beyond 50% of the median latency, and a case where 20% of clients fail to upload entirely.

I. Low-Quality Reference Data (RQ6)

Although TPFed relies on each client's reference dataset to evaluate and select neighbors, real-world data often contain biases, noise, or shifts in distribution. To assess TPFed's resilience, we conducted three experiments where varying proportions (20%, 40%, and 60%) of clients replaced part of their reference data with distorted or adversarial samples.

TABLE VI
PERFORMANCE IMPACT FROM BIASED OR LOW-QUALITY REFERENCE DATA ACROSS DIFFERENT CORRUPTION RATES.

Dataset	20% Corr.	40% Corr.	60% Corr.
MNIST	-0.0066	-0.0104	-0.0189
A-ECG	-0.0011	-0.0023	-0.0055
S-EEG	-0.0005	-0.0013	-0.0037
CIFAR	-0.0008	-0.0015	-0.0024

For instance, in one setup on CIFAR, local noise (potentially privacy-threatening) was injected to simulate suboptimal reference images. Table VI reports the resulting accuracy drops (Δ Accuracy) compared to a baseline with clean, unbiased reference data.

Overall, TPFed maintains robust performance even under 40% corruption, with an accuracy reduction of less than 2% for most datasets. At 60% corruption, some additional degradation becomes apparent, yet the final accuracy remains competitive. This resilience arises from TPFed's multi-level checks—clients not only evaluate peers with reference-data logits (offering a direct performance metric) but also use LSH and rank-based verification to filter out untrustworthy neighbors. However, in extreme cases where reference data are thoroughly compromised or absent, TPFed's all-in-one approach cannot function properly. To mitigate such scenarios, reference samples can be derived from more diverse data sources, augmented periodically, or verified cooperatively across multiple clients. Empirically, our experiments confirm TPFed's tolerance of moderate biases, ensuring stable personalization and training in real-world deployments.

VI. CONCLUSION

This study proposes TPFed, a fully decentralized federated learning framework that integrates inter-client similarity and model quality assessments for neighbor selection, coupled with privacy-preserving knowledge distillation and LSH verification mechanisms. TPFed enhances learning performance, system robustness, and data privacy, making it a promising solution for secure and personalized federated learning in sensitive applications such as e-health and decentralized finance.

Limitations: First, regarding model compatibility, TPFed currently operates best under homogeneous architectures with shared initialization. While neural network permutation invariance may theoretically affect LSH accuracy, our experiments suggest that even in cases where permutation leads to falsely large distances, TPFed's redundancy allows clients

to find other sufficient neighbors, while the subsequent logits verification prevents false positives. Second, regarding system incentives, the blockchain component currently lacks granular punitive mechanisms to penalize malicious actors beyond simple exclusion. Future work will focus on extending TPFed to heterogeneous model settings and integrating game-theoretical incentive designs.

ACKNOWLEDGMENTS

This work was supported by the National Natural Science Foundation of China (U22B2038, 62192784), the Fundamental Research Funds for the Central Universities (510224045), the National Natural Science Foundation of China (U23A20319, 62272054), and the Beijing Nova Program (20230484319).

REFERENCES

- [1] K. Bonawitz, "Towards federated learning at scale: System design," *arXiv preprint arXiv:1902.01046*, 2019.
- [2] H. Wang, Z. Kaplan, D. Niu, and B. Li, "Optimizing federated learning on non-iid data with reinforcement learning," in *IEEE INFOCOM 2020-IEEE conference on computer communications*. IEEE, 2020, pp. 1698–1707.
- [3] Y. Zhang, Y. Bai, and N. Jiang, "Offline learning in Markov games with general function approximation," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, and J. Scarlett, Eds., vol. 202. PMLR, 23–29 Jul 2023, pp. 40 804–40 829. [Online]. Available: <https://proceedings.mlr.press/v202/zhang23a.html>
- [4] Y. Sun, J. Shao, Y. Mao, J. H. Wang, and J. Zhang, "Semi-decentralized federated edge learning with data and device heterogeneity," *IEEE Transactions on Network and Service Management*, vol. 20, no. 2, pp. 1487–1501, 2023.
- [5] E. T. M. Beltrán, M. Q. Pérez, P. M. S. Sánchez, S. L. Bernal, G. Bovet, M. G. Pérez, G. M. Pérez, and A. H. Celdrán, "Decentralized federated learning: Fundamentals, state of the art, frameworks, trends, and challenges," *IEEE Communications Surveys & Tutorials*, 2023.
- [6] X. Zhou, W. Huang, W. Liang, Z. Yan, J. Ma, Y. Pan, I. Kevin, and K. Wang, "Federated distillation and blockchain empowered secure knowledge sharing for internet of medical things," *Information Sciences*, vol. 662, p. 120217, 2024.
- [7] C. Wan, Y. Wang, J. Xu, J. Wu, T. Zhang, and Y. Wang, "Research on privacy protection in federated learning combining distillation defense and blockchain," *Electronics*, vol. 13, no. 4, p. 679, 2024.
- [8] G. Kollias, T. Salonidis, and S. Wang, "Sketch to skip and select: Communication efficient federated learning using locality sensitive hashing," in *International Workshop on Trustworthy Federated Learning*. Springer, 2022, pp. 72–83.
- [9] Q. Chen, Z. Wang, H. Wang, and X. Lin, "Feddual: Pair-wise gossip helps federated learning in large decentralized networks," *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 335–350, 2022.
- [10] A. A. Abdellatif, K. Shaban, and A. Massoud, "Sdcl: A framework for secure, distributed, and collaborative learning in smart grids," *IEEE Internet of Things Magazine*, vol. 7, no. 3, pp. 84–90, 2024.
- [11] N. M. Hijazi, M. Aloqaily, and M. Guizani, "Collaborative iot learning with secure peer-to-peer federated approach," *Computer Communications*, p. 107948, 2024.
- [12] B. Liu, Y. Cai, H. Bi, Z. Zhang, D. Li, Y. Guo, and X. Chen, "Beyond fine-tuning: Efficient and effective fed-tuning for mobile/web users," in *Proceedings of the ACM Web Conference 2023*, ser. WWW '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 2863–2873. [Online]. Available: <https://doi.org/10.1145/3543507.3583212>
- [13] D. Domini, G. Aguzzi, N. Farabegoli, M. Viroli, and L. Esterle, "Proximity-based self-federated learning," *arXiv preprint arXiv:2407.12410*, 2024.
- [14] H. Ye, L. Liang, and G. Y. Li, "Decentralized federated learning with unreliable communications," *IEEE journal of selected topics in signal processing*, vol. 16, no. 3, pp. 487–500, 2022.
- [15] N. Blum, S. Lachapelle, and H. Alvestrand, "WebRTC-realtime communication for the open web platform: What was once a way to bring audio and video to the web has expanded into more use cases we could ever imagine," *Queue*, vol. 19, no. 1, pp. 77–93, 2021.
- [16] L. Rossi and L. Ferretti, "Webdht: browser-compatible distributed hash table for decentralized web applications," in *2022 IEEE 21st International Symposium on Network Computing and Applications (NCA)*, vol. 21. IEEE, 2022, pp. 277–284.
- [17] O. Dib and B. Rababah, "Decentralized identity systems: Architecture, challenges, solutions and future directions," *Annals of Emerging Technologies in Computing (AETiC)*, vol. 4, no. 5, pp. 19–40, 2020.
- [18] X. Deng, J. Liu, L. Wang, and Z. Zhao, "A trust evaluation system based on reputation data in mobile edge computing network," *Peer-to-Peer Networking and Applications*, vol. 13, pp. 1744–1755, 2020.
- [19] Y. Zhao and J. Chen, "A survey on differential privacy for unstructured data content," *ACM Computing Surveys (CSUR)*, vol. 54, no. 10s, pp. 1–28, 2022.
- [20] O. O. Olakanmi and K. O. Odeyemi, "Trust-aware and incentive-based offloading scheme for secure multi-party computation in internet of things," *Internet of Things*, vol. 19, p. 100527, 2022.
- [21] J.-h. Duan, W. Li, D. Zou, R. Li, and S. Lu, "Federated learning with data-agnostic distribution fusion," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 8074–8083.
- [22] D. Makhija, X. Han, N. Ho, and J. Ghosh, "Architecture agnostic federated learning for neural networks," in *International Conference on Machine Learning*. PMLR, 2022, pp. 14 860–14 870.
- [23] X. Lian, C. Zhang, H. Zhang, C.-J. Hsieh, W. Zhang, and J. Liu, "Can decentralized algorithms outperform centralized algorithms? a case study for decentralized parallel stochastic gradient descent," *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [24] Y. Lu, X. Huang, Y. Dai, S. Maharjan, and Y. Zhang, "Blockchain and federated learning for privacy-preserved data sharing in industrial iot," *IEEE Transactions on Industrial Informatics*, vol. 16, no. 6, pp. 4177–4186, 2019.
- [25] D. Li and J. Wang, "Fedmd: Heterogeneous federated learning via model distillation," *arXiv preprint arXiv:1910.03581*, 2019.
- [26] J. Wang, Q. Li, L. Lyu, and F. Ma, "pfedclub: Controllable heterogeneous model aggregation for personalized federated learning," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. [Online]. Available: <https://openreview.net/forum?id=xW6ga9i4eA>
- [27] G. Ye, T. Chen, Y. Li, L. Cui, Q. V. H. Nguyen, and H. Yin, "Heterogeneous collaborative learning for personalized healthcare analytics via messenger distillation," *IEEE Journal of Biomedical and Health Informatics*, vol. 27, no. 11, pp. 5249–5259, 2023.
- [28] S. Kalra, J. Wen, J. C. Cresswell, M. Volkovs, and H. R. Tizhoosh, "Decentralized federated learning through proxy model sharing," *Nature communications*, vol. 14, no. 1, p. 2899, 2023.
- [29] E. Jeong and M. Kountouris, "Personalized decentralized federated learning with knowledge distillation," in *ICC 2023-IEEE International Conference on Communications*. IEEE, 2023, pp. 1982–1987.
- [30] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [31] Y. Ichimaru and G. Moody, "Development of the polysomnographic database on cd-rom," *Psychiatry and clinical neurosciences*, vol. 53, no. 2, pp. 175–177, 1999.
- [32] W. Cai and D. Hu, "Qrs complex detection using novel deep learning neural networks," *IEEE Access*, vol. 8, pp. 97 082–97 089, 2020.
- [33] M. Mourtazaev, B. Kemp, A. Zwiderman, and H. Kamphuisen, "Age and gender affect different characteristics of slow waves in the sleep eeg," *Sleep*, vol. 18, no. 7, pp. 557–564, 1995.
- [34] A. Rechtschaffen, "A manual for standardized terminology, techniques and scoring system for sleep stages in human subjects," *Brain information service*, 1968.
- [35] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.
- [36] K. He, X. Zhang, S. Ren, and J. Sun, "Identity mappings in deep residual networks," in *Computer Vision—ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part IV 14*. Springer, 2016, pp. 630–645.